

Universidad Nacional de Tucumán

Facultad de Ciencias Exactas y Tecnología



Sistema de Simulación de Combate e Inventario para un Videojuego RPG

Binciguerra, Santiago: Licenciatura en Informática
Gomes Opitz, Mariano: Licenciatura en Informática

2026

Patrones de Lenguaje
Departamento Ciencias de Computación

Tabla de Contenidos

1. Descripción del problema.....	2
1.1. Módulo A: Gestión y Creación de Combatientes.....	2
1.2. Módulo B: Mecánica de Combate y Estados Alterados.....	2
1.3. Módulo C: Sistema de Inventario Jerárquico.....	2
1.4. Módulo D: Modificadores Dinámicos de Equipamiento.....	3
1.5. Módulo E: Desacoplamiento de Eventos Globales.....	3
2. Patrones involucrados.....	3
2.1. Factory Method.....	3
2.2. State.....	4
2.3. Composite.....	4
2.4. Decorator.....	4
2.5. Observer.....	5
2.6. Singleton.....	5
2.7. Facade.....	6
3. Diagramas de clases.....	7
3.1. Diagrama principal.....	7
3.2. Diagrama del patrón Factory Method.....	8
3.3. Diagrama del patrón State.....	9
3.4. Diagrama del patrón Composite.....	10
3.5. Diagrama del patrón Decorator.....	11
3.6. Diagrama del patrón Observer.....	12
3.7. Diagrama del patrón Facade.....	13
4. Fuentes y enlaces.....	14

1. Descripción del problema

El objetivo de este proyecto es diseñar e implementar el núcleo lógico (backend) para un sistema de videojuegos de rol (RPG) basado en turnos. El sistema deberá gestionar de forma robusta la creación de personajes, la alteración de sus comportamientos durante el combate, la organización jerárquica de sus pertenencias, la modificación dinámica de su equipamiento y la notificación de eventos hacia sistemas secundarios del juego.

El sistema se divide en cinco módulos principales interconectados:

1.1. Módulo A: Gestión y Creación de Combatientes

El motor debe permitir la aparición de distintos tipos de entidades en el campo de batalla. Inicialmente se contemplan tres clases de personajes seleccionables: **Luchador**, **Bárbaro**, **Arquero**, y **Hechicero**. Cada uno de ellos posee atributos base diferenciados (Fuerza, Percepción, Resistencia, Carisma, Inteligencia, Agilidad, Suerte, y Puntos de Vida). El proceso de inicialización y asignación de estos atributos según la clase elegida debe estar centralizado y desacoplado del flujo principal del combate, permitiendo la incorporación futura de nuevas clases de personajes (como Clérigos, Bardos, Caballeros) sin modificar el código que gestiona la partida.

1.2. Módulo B: Mecánica de Combate y Estados Alterados

Durante el desarrollo de un combate por turnos, los personajes realizan acciones (Atacar, Defender, Usar Habilidad). Sin embargo, la ejecución y efectividad de estas acciones dependerá estrictamente del estado físico o mágico actual del personaje. Un personaje puede encontrarse en uno de los siguientes estados principales:

- **Saludable:** Realiza sus acciones de manera normal y con efectividad plena.
- **Envenenado:** Al inicio de su propio turno, el personaje pierde automáticamente un porcentaje de sus puntos de vida máximos antes de poder actuar.
- **Paralizado:** El personaje pierde completamente la oportunidad de actuar en su turno actual, saltando al siguiente combatiente en la cola.
- **Muerto:** No puede recibir acciones de ataque, no toma turnos y bloquea cualquier intento de modificar sus estadísticas vitales.

El sistema debe cambiar la condición del personaje dinámicamente al recibir ciertos ataques y debe evitar el uso de estructuras condicionales masivas (**if/else** o **switch**) dentro de los métodos del personaje para verificar qué estado posee en ese momento.

1.3. Módulo C: Sistema de Inventario Jerárquico

Cada personaje posee un almacenamiento de pertenencias. El inventario debe ser capaz de manejar tanto objetos individuales simples (**Pociones, Gemas, Armas**) como estructuras contenedoras (**Bolsas de cuero, Cofres pequeños**). Los contenedores pueden albergar en su interior objetos simples u otros sub-contenedores de forma indefinida. El sistema requiere calcular de manera uniforme y transparente dos propiedades fundamentales de cualquier elemento del inventario: el **Peso Total** y el **Valor en Oro**. Si se solicita el peso de un cofre, este debe devolver la suma del peso de su propia estructura más el peso de todo lo que contiene en su interior de forma recursiva.

1.4. Módulo D: Modificadores Dinámicos de Equipamiento

Las armas base (como una **Espada Corta** o un **Arco Largo**) poseen un valor de daño fijo. Sin embargo, el juego debe permitir la aplicación de mejoras sobre dichas armas durante la partida. Por ejemplo, una espada puede imbuirse en fuego (**Modificador de Fuego**, que añade +15 de daño) o afilarse (**Modificador de Filo**, que añade un 10% al daño base). Estas mejoras deben poder apilarse de forma infinita sobre un mismo objeto (por ejemplo, una *Espada Corta con Filo y Fuego*) en tiempo de ejecución, alterando el resultado del método `calcularDano()` sin necesidad de crear clases rígidas para cada combinación posible de armas e incrustaciones.

1.5. Módulo E: Desacoplamiento de Eventos Globales

Ciertas acciones críticas dentro del combate tienen repercusiones en sistemas que no pertenecen a la lógica matemática del juego. Por ejemplo, cuando la salud de un personaje llega a cero (muerte):

- El **Sistema de Registro (Logger)** debe asentar en un archivo de texto los detalles del deceso para auditoría de la partida.

La clase del personaje no debe conocer la existencia ni los métodos de estos sistemas secundarios; simplemente debe emitir un aviso general cuando ocurra el cambio en sus estadísticas vitales.

Además, el sistema debe permitir que en el futuro, la cantidad de estos sistemas aumente y disminuya, por lo que es conveniente trabajar con éstos de una manera general y ordenada.

2. Patrones involucrados

2.1. Factory Method

- **Propósito:** Creación de objetos.

- **Aplicación:** Se define una interfaz o clase abstracta constructora (ej. `CreadorPersonajes`) con un método abstracto `CrearPersonaje()`. Las subclases concretas (`CreadorGuerrero`, `CreadorMago`) implementarán este método para instanciar y parametrizar correctamente el tipo específico de combatiente. Esto permite que el bucle del juego solicite nuevos enemigos o aliados de forma genérica a través de la abstracción, encapsulando las reglas de inicialización de atributos de cada clase de personaje.

2.2. State

- **Propósito:** Comportamiento dependiente del estado.
- **Aplicación:** Se extrae el comportamiento asociado a la condición del personaje y se encapsula en una jerarquía de clases separada bajo la interfaz `EstadoPersonaje`. Las clases concretas (`EstadoSaludable`, `EstadoEnvenenado`, `EstadoParalizado`, `EstadoMuerto`) implementarán de manera particular los métodos `Atacar()` o `AplicarEfecto()`. El personaje mantendrá una referencia privada a su estado actual. Cuando el personaje intente actuar, delegará la ejecución al objeto estado actual (`estadoActual.Atacar()`), eliminando los condicionales anidados y permitiendo transiciones limpias entre estados.

2.3. Composite

- **Propósito:** Estructuras jerárquicas parte-todo.
- **Aplicación:** Se establece una interfaz común llamada `Inventario` que declara las operaciones `GetPeso()` y `GetValor()`. Los objetos simples (como la clase `Item`) actúan como las hojas del árbol e implementan estos métodos devolviendo sus valores directos. Las estructuras contenedoras (como la clase `Contenedor`) actúan como compuestos, manteniendo una lista interna de tipo `Inventario`. Al invocar `GetPeso()` en un contenedor, este recorrerá su lista sumando recursivamente los pesos de sus componentes, tratando a elementos simples y compuestos bajo una misma interfaz unificada.

2.4. Decorator

- **Propósito:** Añadir responsabilidades dinámicamente.
- **Aplicación:** Se define una clase abstracta **ModificadorArma** que implementa la interfaz base **Arma** y contiene una referencia agregada a otro objeto de tipo **Arma**. Los decoradores concretos (**DecoradorFuego**, **DecoradorFilo**, etc) heredan de este modificador base. Al sobrescribir el método **CalcularDano()**, cada decorador ejecutará el cálculo del arma interna que está envolviendo y le añadirá su propia bonificación. Esto permite estructurar envolturas en tiempo de ejecución (**new DecoradorFuego(new DecoradorFilo(new Arma()))**), extendiendo el comportamiento del objeto individual de forma transparente.

2.5. Observer

- **Propósito:** Notificación de eventos a archivo de texto.
- **Aplicación:** La clase **Personaje** asume el rol del **Sujeto Concreto**, heredando de una estructura que mantiene una lista de suscriptores a través de la interfaz **Observador**. El componente periférico (**SistemaLogros**) implementa dicha interfaz **Observador** y se registran eventos del personaje. Cuando la vida del personaje se altera o llega a cero, se equipa una nueva arma, cambia de estado, este invoca de manera automática al método **NotificarObservadores()**, provocando que el/los sistemas adheridos ejecuten su respectiva lógica de respuesta de forma totalmente desacoplada.

2.6. Singleton

- **Propósito:** Garantizar que una clase tenga una única instancia en todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global y unificado a dicha instancia.
- **Aplicación:** Este patrón se implementa de manera estratégica en dos módulos del sistema para resolver problemas de eficiencia en la asignación de memoria y colisiones en la gestión de recursos del hardware:
 - **En los Estados Concretos (Integración con el Patrón State):** Las clases derivadas que representan las condiciones físicas o mágicas de los combatientes (**EstadoSaludable**, **EstadoEnvenenado**, **EstadoParalizado**, **EstadoMuerto**) no poseen variables de instancia propias; su única función es encapsular algoritmos de comportamiento que operan directamente sobre el personaje que reciben por parámetro. Al transformar cada estado concreto en un Singleton con un constructor privado y un método estático de acceso, se evita la creación masiva y la subsecuente destrucción de objetos idénticos cada vez que

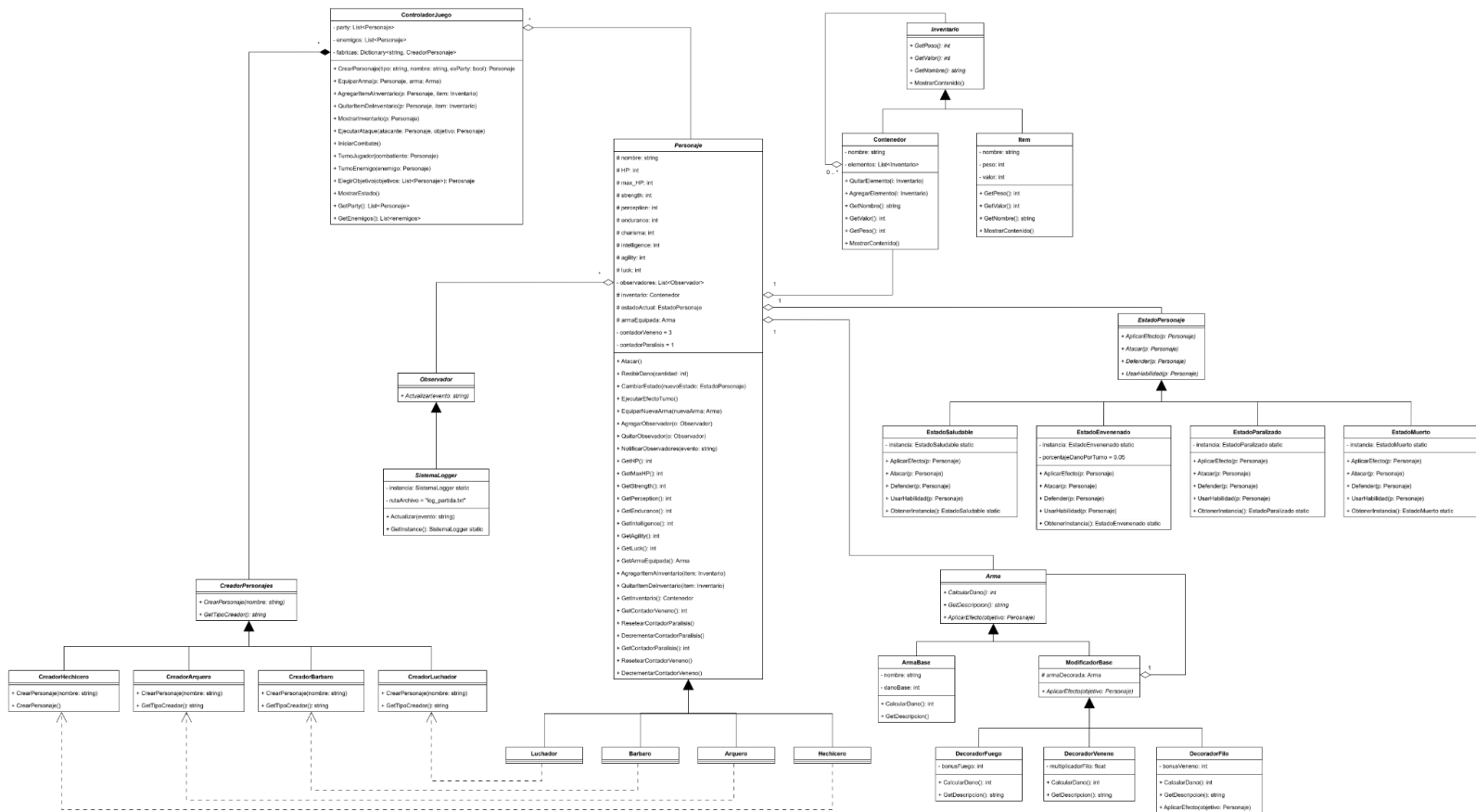
una entidad cambia de condición. Todos los personajes de la simulación comparten la misma instancia única de cada estado en memoria, reduciendo la fragmentación y optimizando el rendimiento del bucle principal del juego.

- **En los Sistemas Observadores (Integración con el Patrón Observer):** Los componentes periféricos encargados de reaccionar a los eventos del juego manejan recursos del entorno que son intrínsecamente únicos. Específicamente, el **SistemaLogger** administra el acceso exclusivo a un archivo de texto en el almacenamiento físico. Al definir los observadores como Singletons, se garantiza que no existan múltiples instancias intentando escribir en el mismo archivo simultáneamente (lo que provocaría un bloqueo por concurrencia), proveyendo un punto de acceso seguro desde cualquier sección del motor lógico.

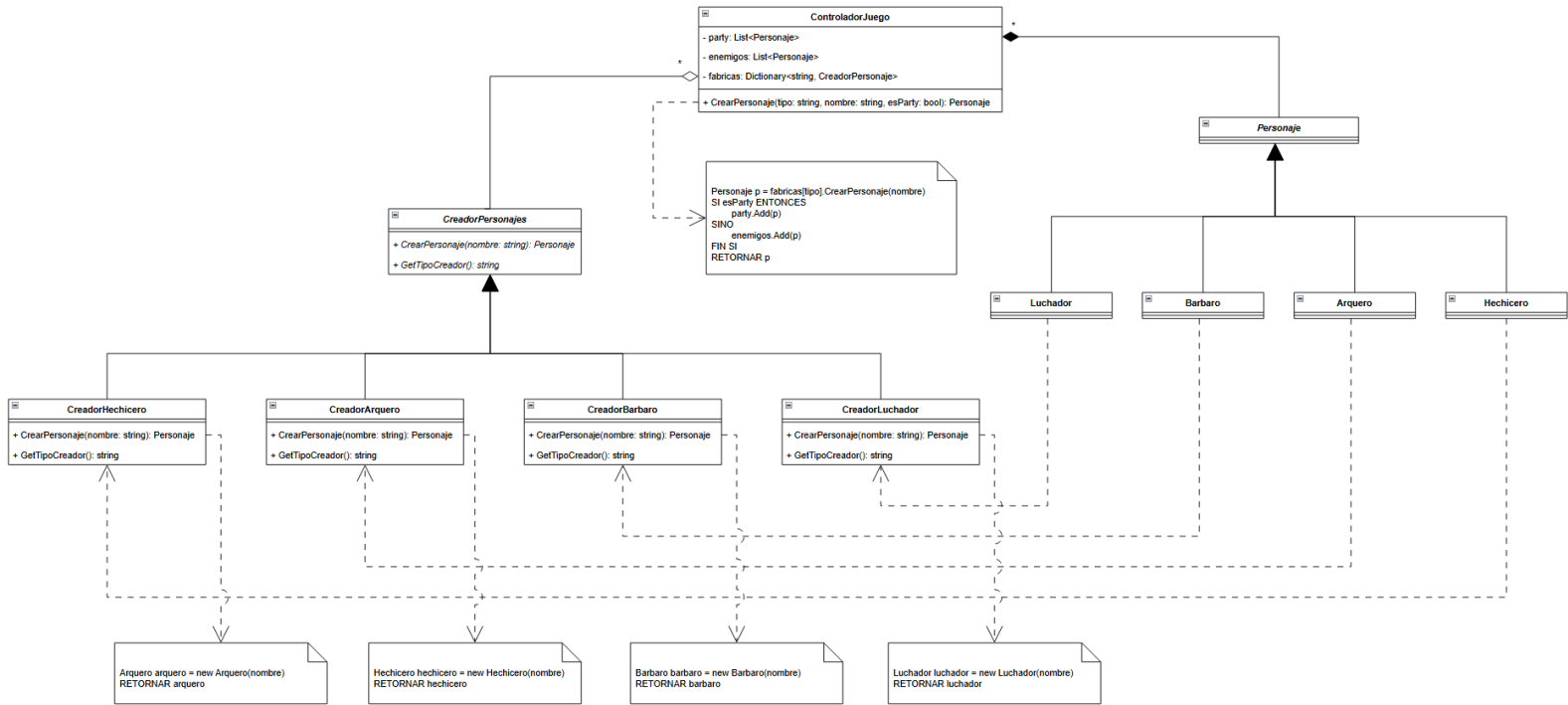
2.7. Facade

- **Propósito:** Conectar todas las funcionalidades lógicas y permitir que el usuario interactúe con el sistema de manera limpia
- **Aplicación:** Previene exponer directamente la lógica del sistema (creadores, estados, decorados) a la interfaz de usuario, evitando generar un alto acoplamiento y que el sistema mismo sea difícil de mantener. La clase **ControladorJuego** actúa como el director del sistema, teniendo la responsabilidad de instanciar los componentes iniciales, coordinar las acciones, y mantener el estado global de la partida. Se encarga tanto de gestionar las fábricas (Creadores concretos), las entidades, los observadores, como también iniciar partidas, ejecutar ataques, consultar inventarios, etc.

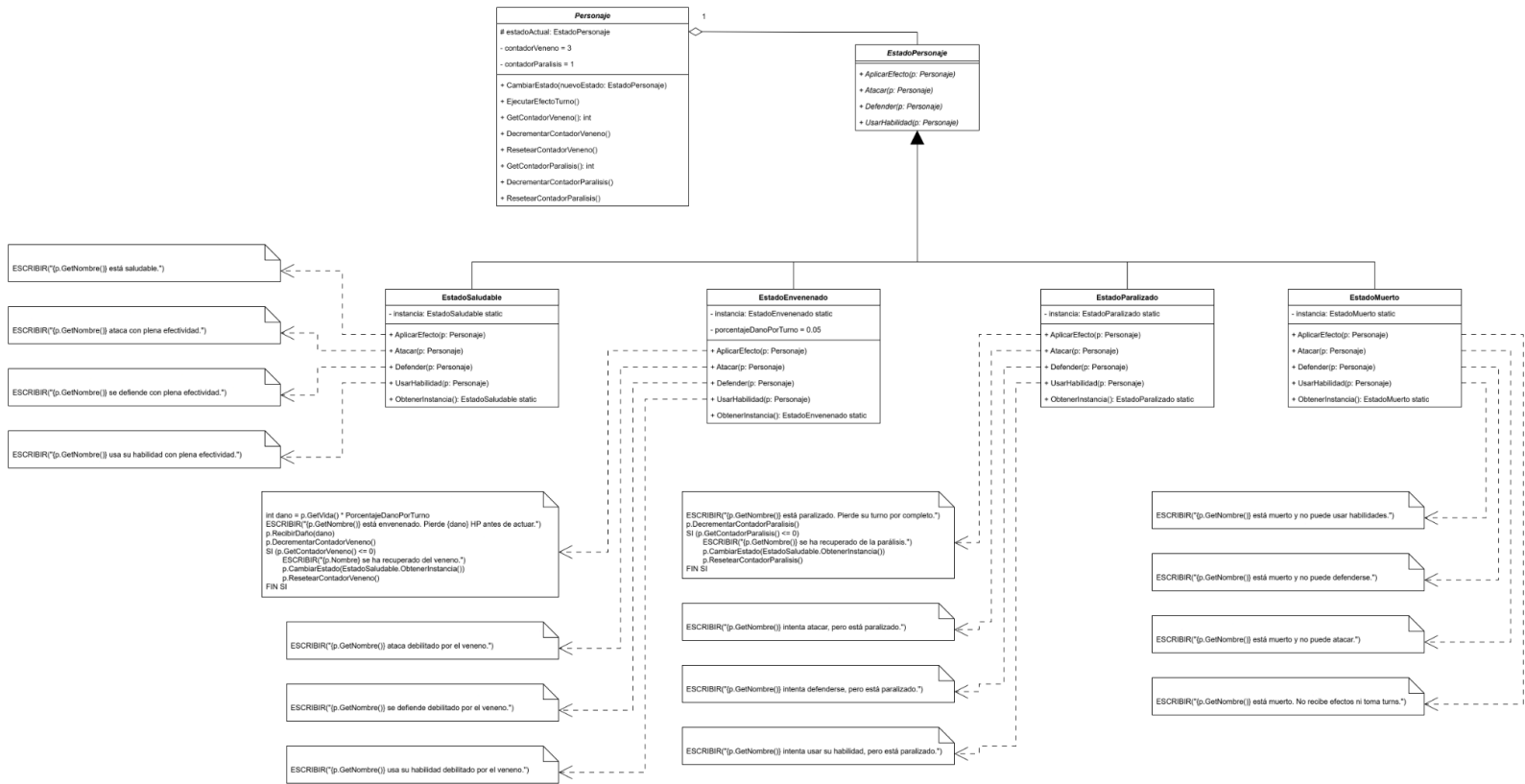
3.1. Diagrama principal



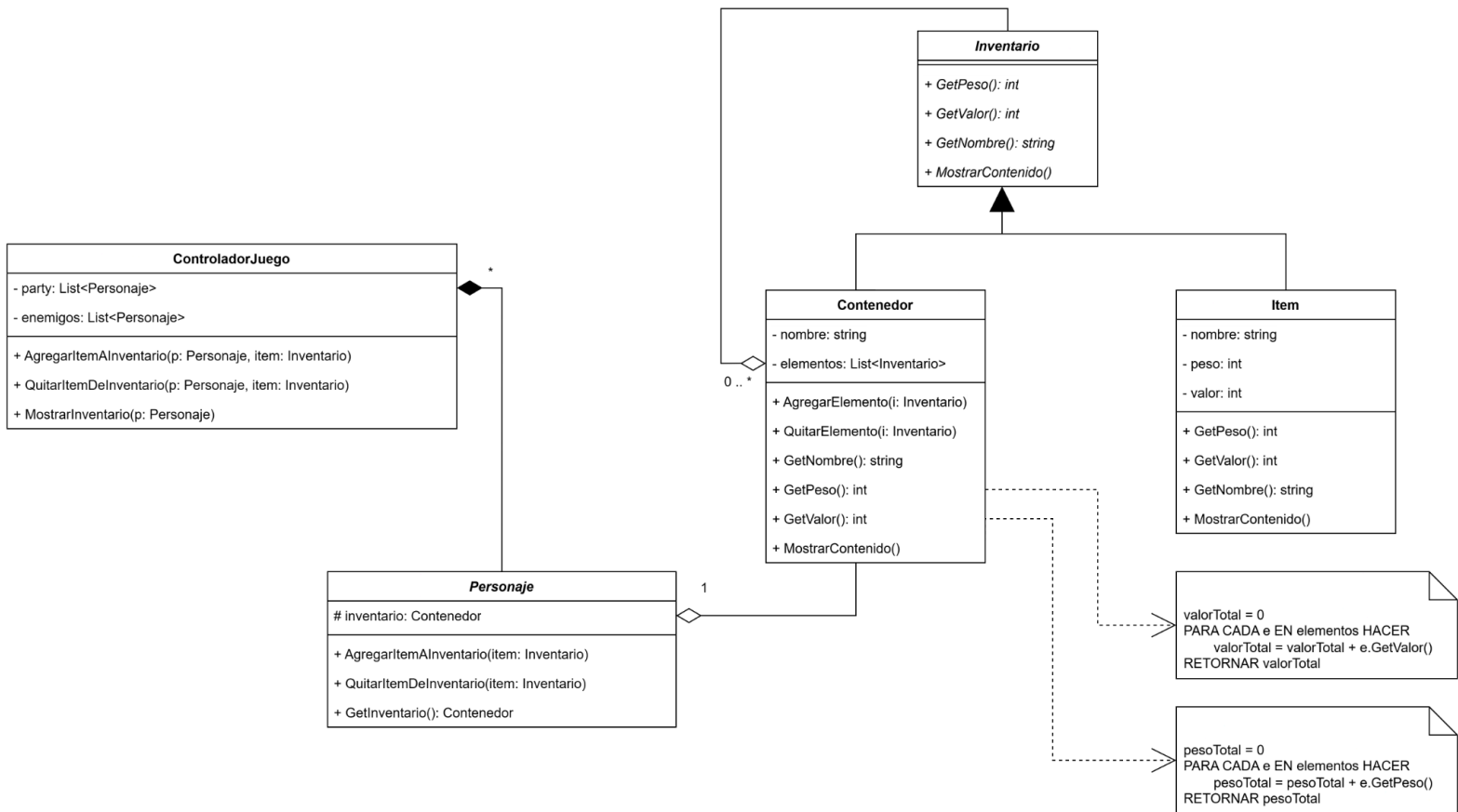
3.2. Diagrama del patrón Factory Method



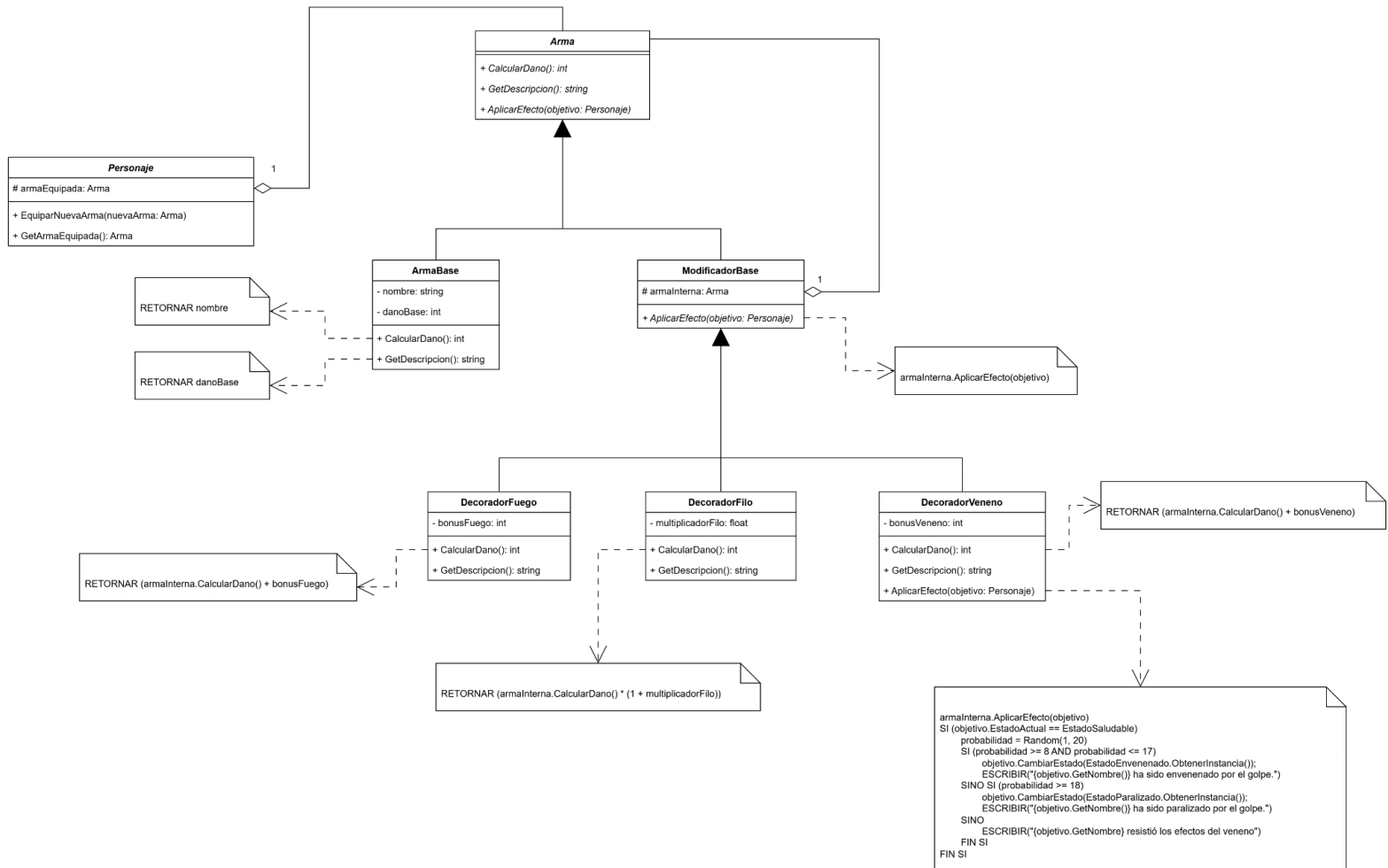
3.3. Diagrama del patrón State



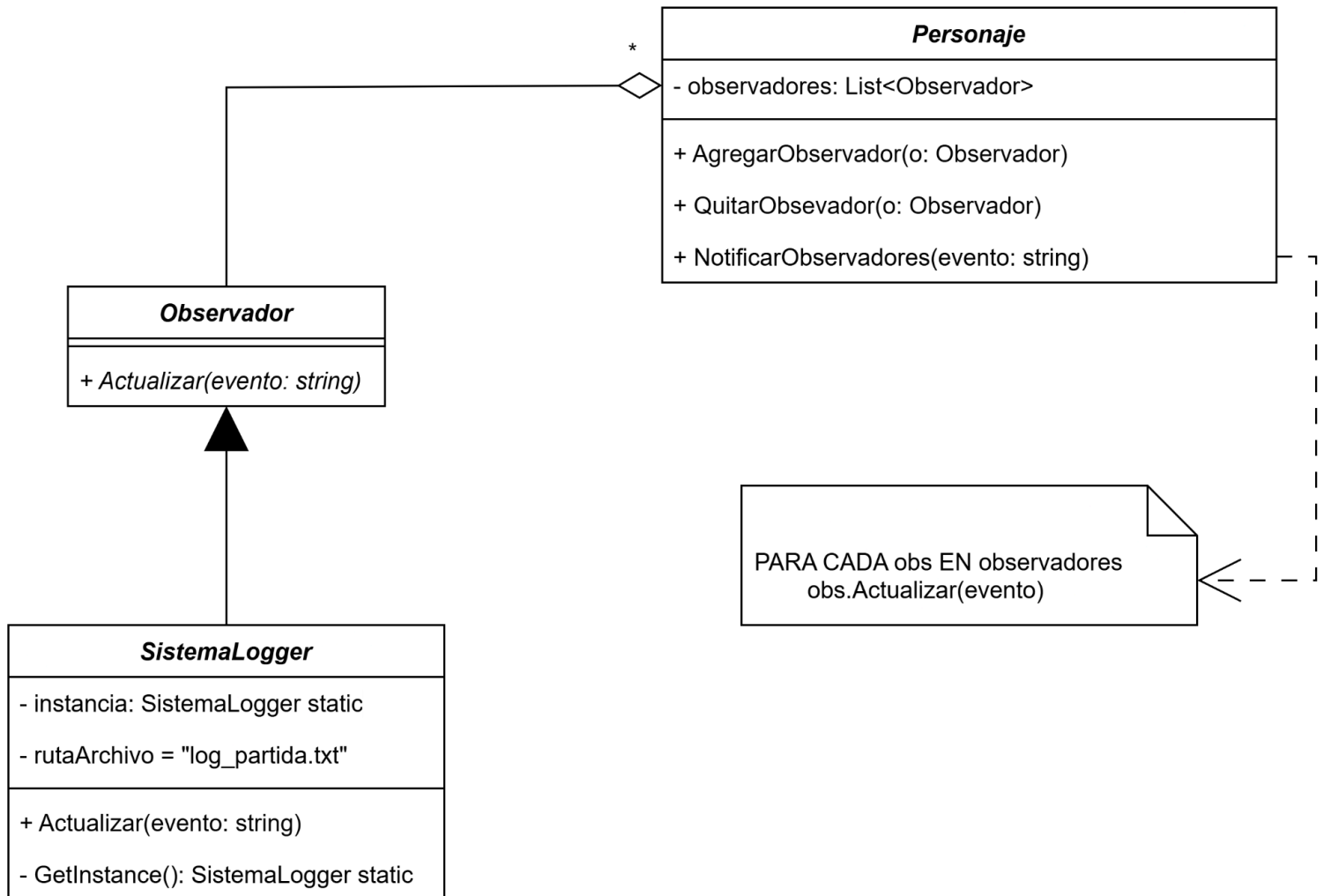
3.4. Diagrama del patrón Composite



3.5. Diagrama del patrón Decorator



3.6. Diagrama del patrón Observer



3.7. Diagrama del patrón Facade

ControladorJuego
- party: List<Personaje> - enemigos: List<Personaje> - fabricas: Dictionary<string, CreadorPersonaje>
+ CrearPersonaje(tipo: string, nombre: string, esParty: bool): Personaje + EquiparArma(p: Personaje, arma: Arma) + AgregarItemAlInventario(p: Personaje, item: Inventario) + QuitarItemDelInventario(p: Personaje, item: Inventario) + MostrarInventario(p: Personaje) + EjecutarAtaque(atacante: Personaje, objetivo: Personaje) + IniciarCombate() + TurnoJugador(combatiente: Personaje) + TurnoEnemigo(enemigo: Personaje) + ElegirObjetivo(objetivos: List<Personaje>): Perosnaje + MostrarEstado() + GetParty(): List<Personaje> + GetEnemigos(): List<enemigos>

4. Fuentes y enlaces

- Github. Repositorio con diagramas y código del proyecto.
<https://github.com/Divitrion/Proyecto-Patrones>
- Refactoring Guru. Design Patterns. The Catalog of Design Patterns.
<https://refactoring.guru/design-patterns/catalog>